

I/O

国际化 (i18n) 与字符编码

- ASCII用 7 位表示 128 个字符，扩展 ASCII 使用 8 位表示 256 个字符
- GB代表国标，GBK（国标扩展）扩展GB2312，用于编码更多字符
- Unicode（统一码、万国码、单一码）
 - 每个字符对应一个称为“码点code point”的东西（例如：A的码点是U+0041）
 - Unicode包含1,114,112个码点，范围为0₁₆到10FFFF₁₆
- 编码方案
 - 编码方案遵循Unicode标准并定义码点在内存中的存储方式
 - 不同的编码方案将Unicode码点封装为字节，例如UTF-8、UTF-16、UTF-32

character	encoding	bits
A	UTF-8	01000001
A	UTF-16	00000000 01000001
A	UTF-32	00000000 00000000 00000000 01000001
あ	UTF-8	11100011 10000001 10000010
あ	UTF-16	00110000 01000010
あ	UTF-32	00000000 00000000 00110000 01000010

- UTF-8 【“Unicode变换格式-8位字节”的缩写】
 - 兼容ASCII码表
 - 字符按Unicode值用1到4字节不等的长度进行编码
 - 小值：1字节，例如“T”的UTF-8编码是“01010100”
 - 大值：2、3 或4字节，例如“汉”的UTF-8编码是“11100110 10110001 10001001”
 - Unicode码点的自由比特（以二进制形式）： $a_0, a_1, a_2, \dots$
 - 开头的比特指示用了多少字节。
 - 以0开头的字节：单字节字符
 - 以110、1110或11110开头的字节：多字节字符的起始
 - 以10开头的字节：多字节字符的续字节
 -

Character Range	Encoding
0...7F	0a ₆ a ₅ a ₄ a ₃ a ₂ a ₁ a ₀
80...7FF	110a ₁₀ a ₉ a ₈ a ₇ a ₆ 10a ₅ a ₄ a ₃ a ₂ a ₁ a ₀
800...FFFF	1110a ₁₅ a ₁₄ a ₁₃ a ₁₂ 10a ₁₁ a ₁₀ a ₉ a ₈ a ₇ a ₆ 10a ₅ a ₄ a ₃ a ₂ a ₁ a ₀
10000...10FFFF	11110a ₂₀ a ₁₉ a ₁₈ 10a ₁₇ a ₁₆ a ₁₅ a ₁₄ a ₁₃ a ₁₂ 10a ₁₁ a ₁₀ a ₉ a ₈ a ₇ a ₆ 10a ₅ a ₄ a ₃ a ₂ a ₁ a ₀

A Chinese character: 汉
 its Unicode value: U+6C49
 convert 6C49 to binary: 01101100 01001001

To computer, its simply 0110110001001001
 (don't know whether its 1 or 2 character)

1st Byte	2nd Byte	3rd Byte	4th Byte	Number of Free Bits
0xxxxxx				7
110xxxx	10xxxxxx			(5+6)=11
1110xxxx	10xxxxxx	10xxxxxx		(4+6+6)=16
11110xxx	10xxxxxx	10xxxxxx	10xxxxxx	(3+6+6+6)=21

Add header to free bits

Header	Place holder	Fill in our Binary	Result
1110	xxxx	0110	11100110
10	xxxxxx	110001	10110001
10	xxxxxx	001001	10001001

11100110 10110001 10001001

- UTF-16
 - 不兼容ASCII码表
 - 最少用2字节，不能用3字节，只能用2或4字节
- UTF-32
 - 不兼容ASCII码表
 - 总是用4字节
- Java char类型：
 - Java的char基本类型和String类本质上使用UTF-16编码来表示文本。
 - 在Java中，char是一个16位数据类型，可以表示基本多文种平面（BMP）中的字符，包括从U+0000到U+FFFF的Unicode码点。
 - int与char之间的转换对应Unicode映射。

```
int v1 = 0x2744; // 十六进制
System.out.printf("%d\n", v1); // 10052（十进制）
System.out.printf("%c\n", v1); // *
```

- 超出U+0000到U+FFFF（比如U+1F332“🌲”）的字符，表示时需要多个char。这些字符作为两个char值存储，称为代理对（surrogate pair）。

```
char c1 = '*';
char c2 = '🌲'; // 错误，字符字面量中字符过多

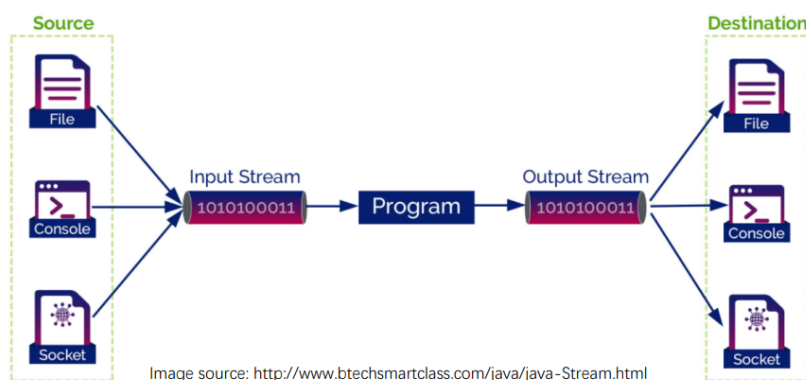
char[] snow = Character.toChars(0x2744); // 长度为1
char[] tree = Character.toChars(0x1F332); // 长度为2

System.out.println(snow); // *
System.out.println(tree); // 🌲
```

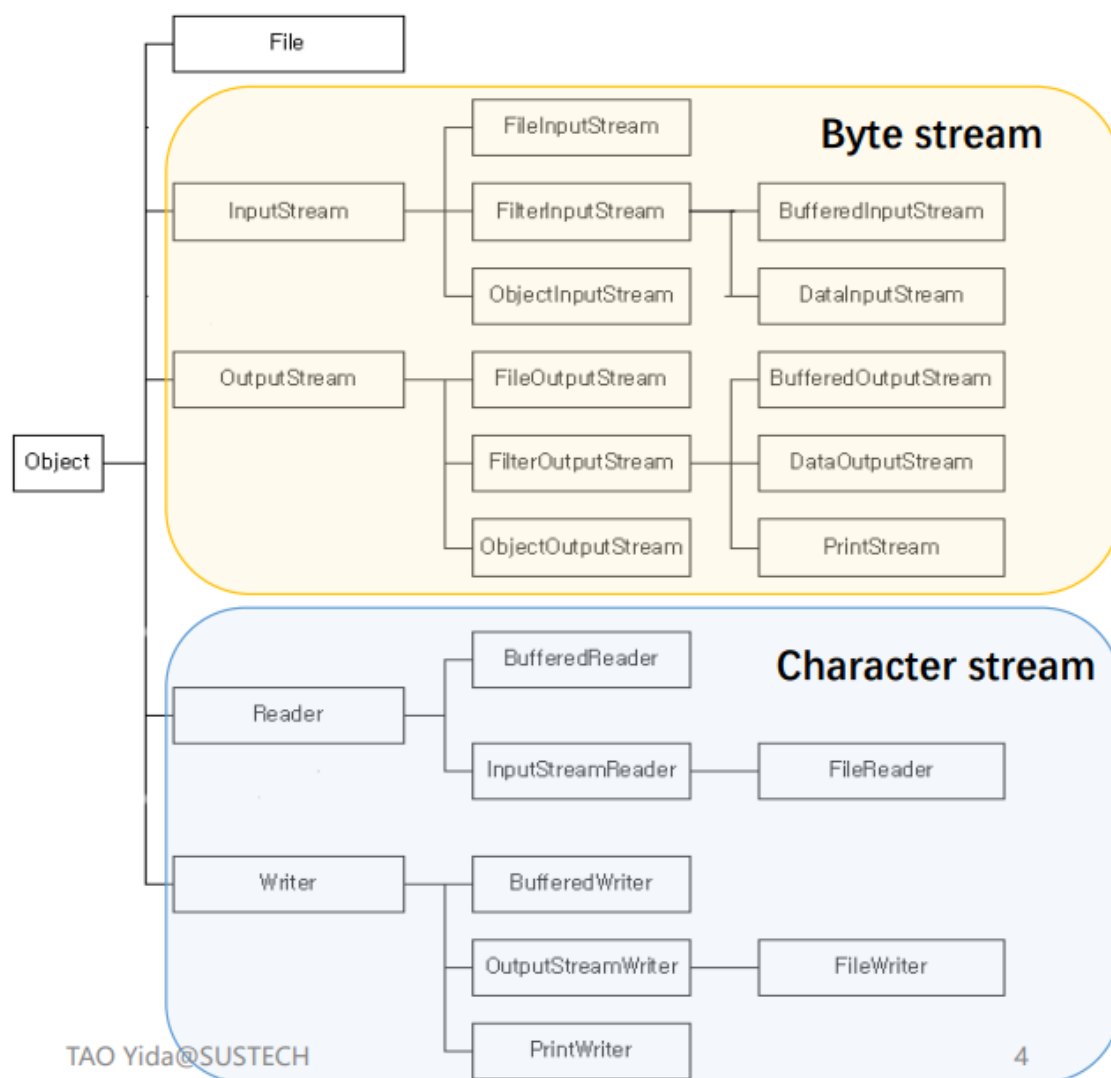
• 字节流与字符流

- Java I/O 流
 - 流是一种连续的数据流，可以顺序访问（不像数组，可以通过索引来回移动）。

- 流作为数据容器，与一个数据源和一个数据目的地连接。



- 字节流与字符流



字节流 (Byte Stream)

- 输入流 (Input stream)：可以读取字节序列的对象
- 输出流 (Output stream)：可以写入字节序列的对象
- 字节流不方便处理以Unicode存储的信息

字符流 (Character Stream)

- 独立的层级提供了从Reader和Writer继承的类，用于处理Unicode字符

- 这些类提供了基于char值的读和写操作，而不是字节值
- InputStream和OutputStream、Reader和Writer都是抽象类
- InputStream或Reader的子类必须实现read()方法
- OutputStream或Writer的子类必须实现write()方法
-

FileInputStream Used for reading streams of raw bytes

```
public void readFile() throws IOException {
    try (InputStream input = new FileInputStream("src/test.txt")) {
        int n;
        while ((n = input.read()) != -1) {
            System.out.println(n);
            ((char)n).打印汉字
        }
    }
}
```

Reading 1 byte a time until there is no more data (-1)

What is the output when test.txt contains the text "Hello World"? (e.g., file encoding is UTF-8)

72 101 108 108 111 32 87 111 114 108 100

- What if test.txt contains “计算机系统” ?

```
try (InputStream input = new FileInputStream("src/test.txt")) {
    int n;
    while ((n = input.read()) != -1) {
        System.out.print(" " + n);
    }
}
```

If file encoding is UTF-8

232 174 161 231 174 151 230 156 186
231 179 187 231 187 159

In UTF-8, normal Chinese characters often take 3 bytes

If file encoding is GBK

188 198 203 227 187 250 207 181 205 179

1 Chinese Character requires more than 1 byte to store (2 bytes for GBK encoding)

GBK encoding for 计: BCC6

- How to get meaningful characters? Can we directly cast bytes to char?

```
try (InputStream input = new FileInputStream("src/test.txt")) {
    int n;
    while ((n = input.read()) != -1) {
        System.out.print((char)n);
    }
}
```

Works only for "Hello World" in which each character can be encoded using only 1 byte
Not working for "计算机系统", which takes 2 bytes (GBK) or 3 bytes (UTF8) for 1 character

- FileReader:

- 用于读取字符流（而不是字节流）
- 问题：数据依然是以0和1（二进制流）读取的，如何确定对应的字符？
- 回答：我们需要指定编码方案（encoding scheme）。如果没有指定，就使用默认的编码方案。

- What if the input txt contains “计算机系统” and has UTF-8 file encoding? (consistent with my Java default encoding UTF-8)

```
try (Reader reader = new FileReader("src/io/sample2.txt")) {
    int n;
    while ((n = reader.read()) != -1) {
        System.out.printf("%d %c\n", n, n);
    }
}
```

35745 计
31639 算
26426 机
31995 系
32479 统

Return the read char as an integer (range 0 to 65535 or 2¹⁶) 计: U+8BA1

- What if the input txt contains “计算机系统” and has GBK file encoding? (inconsistent with my Java default encoding UTF-8)

```
try(Reader reader = new FileReader( fileName: "src/io/sample3.txt")){
    int n;
    while((n=reader.read())!=-1){
        System.out.printf("%d %c\n", n, n);
    }
}
```

65533 ✦
65533 ✦
65533 ✦
65533 ✦
65533 ✦
1013 €
883 τ

Malformed UTF-8 bytes are replaced by default string;
GBK bytes that happen to be valid UTF-8 bytes are mapped to different characters.

- What if the input txt contains “计算机系统” and has GBK file encoding? (inconsistent with my Java default encoding UTF-8)

```
try(Reader reader = new FileReader( fileName: "src/io/sample3.txt", Charset.forName("gb2312"))){
    int n;
    while((n=reader.read())!=-1){
        System.out.printf("%d %c\n", n, n);
    }
}
```

35745 计
31639 算
26426 机
31995 系
32479 统

Set FileReader encoding to be consistent with the file encoding

- Java默认编码：

- Java系统/平台默认编码
 - JVM启动时的默认编码（即，用于决定某个字符对应的字节）
 - 可能与操作系统和语言设置不同（例如中文操作系统上常见的GBK）
 - 可以更改（通过环境变量、IDE、代码等）
- 文件编码
 - 独立于Java
 - 可以更改
 - 用Java读取文件时，Java的默认编码和文件的实际编码应保持一致

- InputStream转Reader

```
// 创建FileInputStream
InputStream input = new FileInputStream("src/test.txt");
// 通过指定编码，转换为FileReader（其实底层就是InputStreamReader）
Reader reader = new InputStreamReader(input, "UTF-8");
```

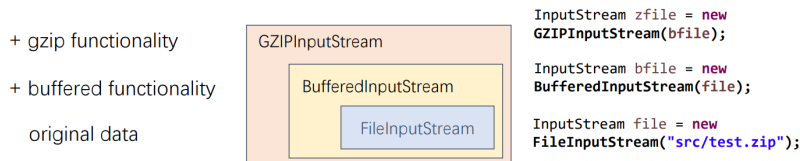
- 用 `FileInputStream` 打开文件，得到字节流（`InputStream`，直接读取字节，无任何字符编码概念）。
- 用 `InputStreamReader` 把这个字节流“包裹”起来，并且按照你指定的字符编码进行解码，转成字符流（Reader）。
- OutputStream和Writer也是同样的模式。

• 组合流过滤器 FilterInputStream

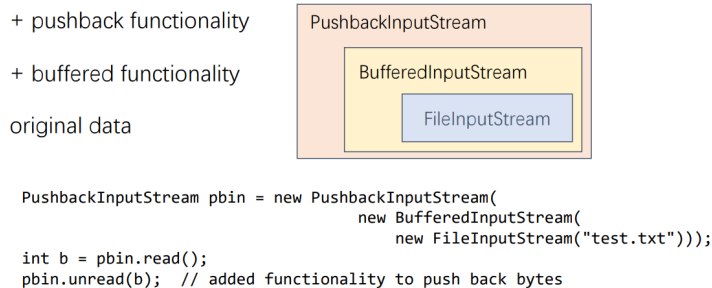
- 包含了其他InputStream，作为其基础数据源
- FilterInputStream的子类在原有流之上提供附加功能
- 直接已知的子类有：
 - BufferedInputStream（带缓冲功能）
 - DataInputStream（处理原始数据类型）

- DigestInputStream (带信息摘要)
- InflaterInputStream (解压)
- LineNumberInputStream (行号)
- PushbackInputStream (回退功能)

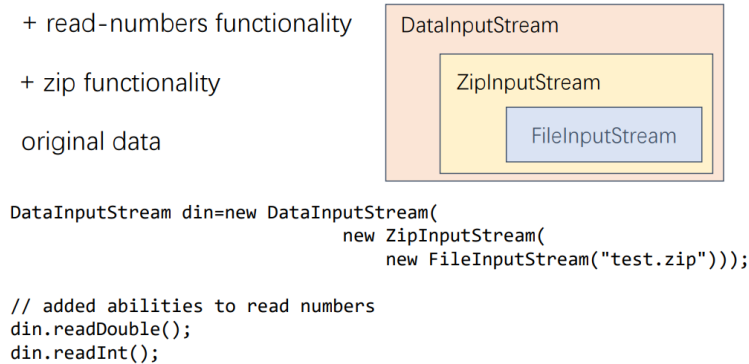
FilterInputStream Example I



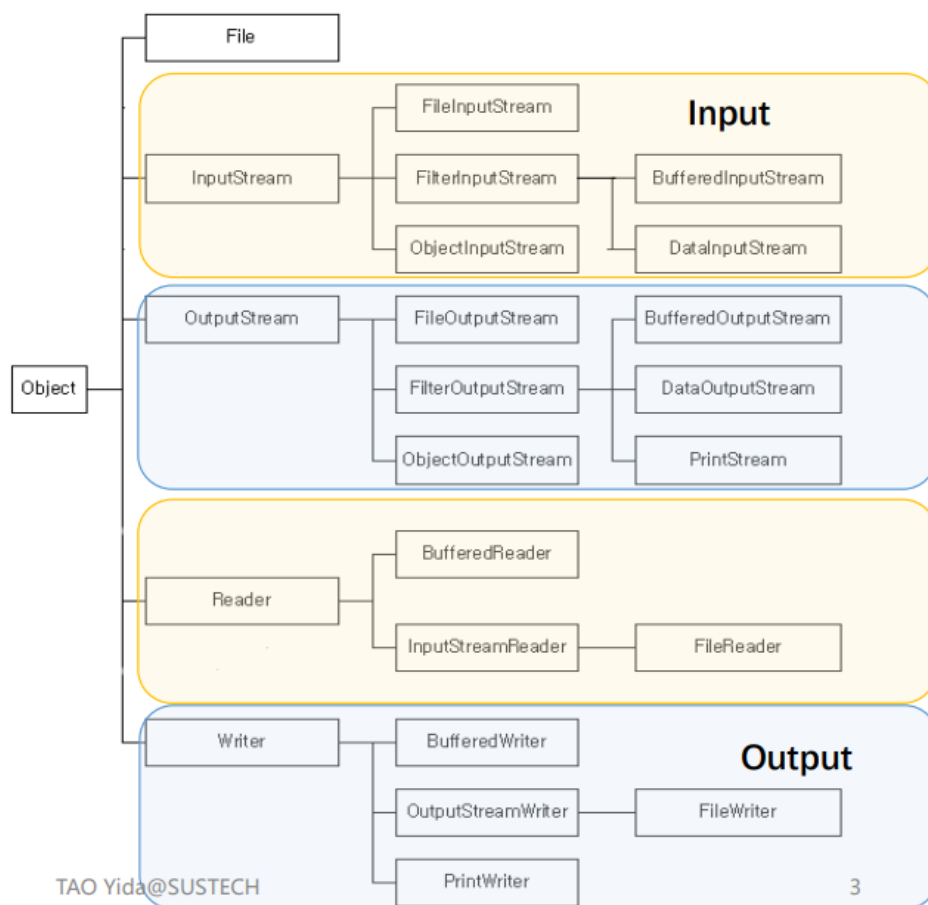
FilterInputStream Example II



FilterInputStream Example III



- 读写文本输入/输出 Reading/Writing Text Input/Output
 - 输入和输出



- 当我们进行I/O操作时，通常处理的是人类可读的文本而不是二进制数据。
- Java 提供了两个API帮助手工处理文本I/O：
 - 扫描（Scanning）：有助于将格式化输入分解为各个标记，并根据数据类型转换单个标记（`java.util.Scanner`）。
 - 格式化（Formatting）：将数据组装成格式良好且易于阅读的人类可读形式（`java.io.PrintWriter`）。
- 使用Scanner读取文本文件
 - 处理任意文本最简单的方法是使用 `Scanner` 类，它可以从输入流中获取输入。
 - `Scanner(InputStream in)`：根据指定的输入流构造一个`Scanner`对象。
 - `String nextLine()`：读取输入的下一行。
 - `String next()`：读取下一个单词（以空白符分隔）。
 - `int nextInt()`：读取下一个整数。
 - `double nextDouble()`：读取并转换下一个表示整数或浮点数的字符序列。
 - `boolean hasNext()`：检测是否有下一个单词。
 - `boolean hasNextInt()`：检测下一个字符序列是否为整数。
 - `boolean hasNextDouble()`：检测下一个字符序列是否为整数或浮点数。
- 使用PrintWriter写入文本文件
 - 想往`PrintWriter`写内容时，可以用与`System.out`一致的`print`, `println`, `printf`方法。
 - 你可以用这些方法打印数字、字符、布尔值、字符串和对象。

```

PrintWriter out = new PrintWriter("employee.txt", "UTF-8");
String name = "Harry Hacker";
double salary = 75000;
out.print(name);
out.print(' ');
out.println(salary);

```

• 命令行输入/输出 I/O from Command Line

- 程序通常从命令行运行，并在命令行环境中与用户交互。
- Java支持这种交互方式有两种：
 - 标准流（Standard Streams）（经常用，比如System.out）
 - 控制台（Console，更高级，比如System.console()）

• The System类

- System类是Object类的子类
- 这是一个final类，不能被其他类继承

```

public final class System extends Object {
    // 这个类不可实例化
    private System() {}
}

```

- 它的构造方法是private的，不能创建其实例
- `System.out.println("Hello World!");`

• 标准流（Standard Streams）

- 标准流从键盘读取输入并将输出写到屏幕上
- Java平台支持三种标准流：

修饰符和类型	字段名	字段和说明
static PrintStream	err	标准错误输出流
static InputStream	in	标准输入流
static PrintStream	out	标准输出流

• System.in

- 标准输入，通常用于读取键盘输入
- System.in是字节流，没有字符流特性。如果要把标准输入当成字符流用，要用InputStreamReader包裹System.in

```

                                Decorator      To Reader      InputStream
BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
String str = "";
while (!str.equals("quit")) {
    str = br.readLine();
    System.out.println(str);
}

```


- **System.out**

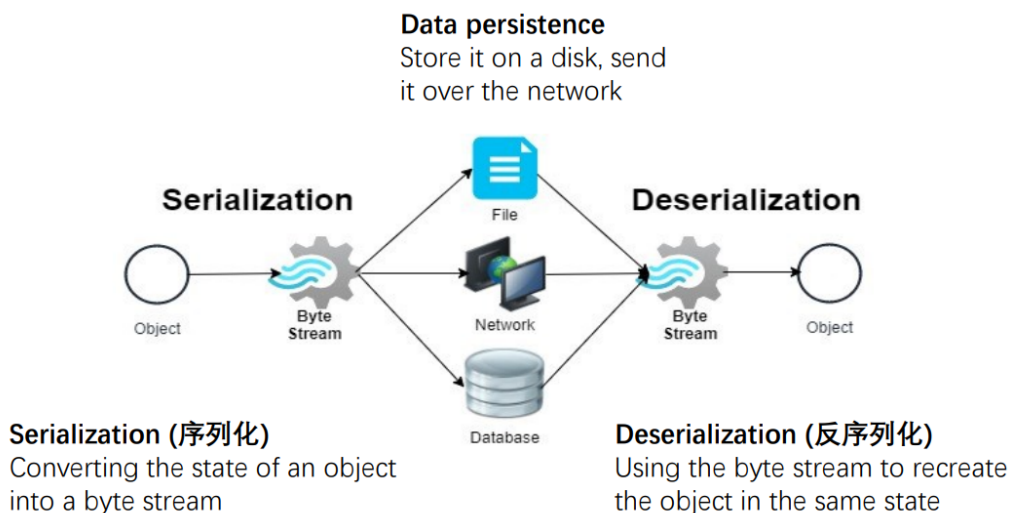
- System.out被定义为PrintStream对象。
- 虽然本质上是字节流，但PrintStream内部用了字符流对象，实现了很多字符流的特性（PrintWriter同理）。
 - print 和 println以标准方式格式化并输出单个值。
 - format可以根据格式化字符串格式化任意数量的数据，支持精确控制输出样式。
- 可以使用 setOut() 将输出 重定向 到其他资源

```
// 构造一个带指定文件的新的 PrintStream
PrintStream out = new PrintStream(new File("src/sysout.txt"));
// 将标准输出从控制台重新分配到文件
System.setOut(out);
// 这将写入到文件
System.out.println("Where am I?");
```

- **File**

- **持久化 & 序列化 Persistence and Serialization**

- Java 程序中创建的对象存在于内存中；当对象不再被使用时，这些对象会被垃圾回收器移除。
- 数据持久化指的是：数据在创建它的进程结束后仍然存在。可以在不重新执行程序达到该状态的情况下，复用这些数据。
-



- **数据持久化 (Persistence)**：把数据存储到磁盘上，或者通过网络发送出去。
- **序列化 (Serialization)**：把内存中的对象（比如Java中的对象），转化成为一种可以存储或传输的字节序列（byte stream）。
- **反序列化 (Deserialization)**：使用字节流还原出与原来状态一致的对象。
- **Serializable 接口**：
 - 类需要实现 serializable 接口，实例才能被序列化或反序列化。

- serializable 接口被称为标记接口（marker interface）或标签接口（tagging interface），就像是在类上加一个标记，编译器和JVM看到这个标记，就知道这个类的对象可以被序列化。
 - serializable 接口是一个空接口，没有任何方法或字段。
 - 实现 serializable 接口的类不需要实现任何方法。

```
Student student = new Student("Alice", "CS", 20);

// Setup where to store the byte stream
FileOutputStream fos = new FileOutputStream("student.ser");
ObjectOutputStream oos = new ObjectOutputStream(fos);

// Serialization
oos.writeObject(student);

oos.close();
```

```
//Setup where to read the byte stream
FileInputStream fis = new FileInputStream("student.ser");
ObjectInputStream ois = new ObjectInputStream(fis);

// Deserialization
Student student2 = (Student) ois.readObject(); // down-casting

ois.close();
```

```
class Student implements Serializable {
    String name;
    String dept;
    int age;

    public String getName() {
        return name;
    }

    public String getDept() {
        return dept;
    }

    public int getAge() {
        return age;
    }

    public Student(String name, String dept, int age) {
        this.name = name;
        this.dept = dept;
        this.age = age;
    }
}
```

- 创建学生对象：Student student = new Student("Alice", "CS", 20);
- 序列化过程：
 - 设置保存字节流的位置（例如 student.ser 文件）。
 - ObjectOutputStream oos = new ObjectOutputStream(fos);
 - 执行 oos.writeObject(student); 把对象写到文件里（序列化）。
 - 关闭流。
- 反序列化过程：
 - 设置读取字节流的位置（同样是 student.ser 文件）。
 - ObjectInputStream ois = new ObjectInputStream(fis);
 - 通过 ois.readObject() 读取对象，并进行强制类型转换。
 - 关闭流。
- 默认序列化机制：
 - ObjectOutputStream 会递归地遍历对象的所有字段，并保存它们的内容。
 - 如果你不希望某个字段被序列化，可以用 transient 关键字标记该字段。
 - 每个对象都会分配一个唯一的序列号（serial ID），有助于在类结构变化时保持兼容性。
- 处理文件Working with Files
 - java.io（JDK 1.0）是基于流的：
 - 主要用于读写字节流或字符流序列。

- 适合I/O流处理，但对于文件系统管理较弱。
- java.io 结合了数据和行为：
 - File 对象既表示文件又提供操作文件的方法。
 - 表示和行为混在一起，不利于扩展和维护。
- java.nio (JDK 7) 是基于文件系统的：
 - 把文件系统本身作为抽象，包括路径、属性、所有权和目录等操作。
 - 更贴合操作系统实际的存储管理方式。
 - java.nio.file 分离了关注点（解耦了对象职责）：

- Path 表示文件或目录的位置（你想操作的对象）。 `import java.nio.file.Path;`

```
Path p1 = Path.of("/home/user/images/pic.png");
```

```
Path p2 = Paths.get("resources");
```

- Files 提供一系列静态方法用于操作 Path（你如何对其操作）。 `import java.nio.file.Files`

```
Path path = Path.of("info.txt");

// 判断文件是否存在
Files.exists(path);

// 复制文件
Files.copy(path, Path.of("backup/info.txt"));

// 读取文件所有内容
Files.readAllLines(path);
BufferedReader reader = Files.newBufferedReader(path);

// 写入内容到文件
Files.write(path, List.of("Hello", "World"));

// 删除文件
Files.delete(path);
```

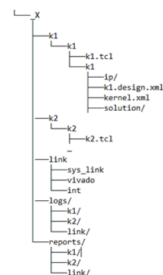
- Files.walk 以深度优先的方式进入所有子目录

Recursive Directory Traversal

`Files.walk`
Enter all subdirectories in a [depth-first](#) manner

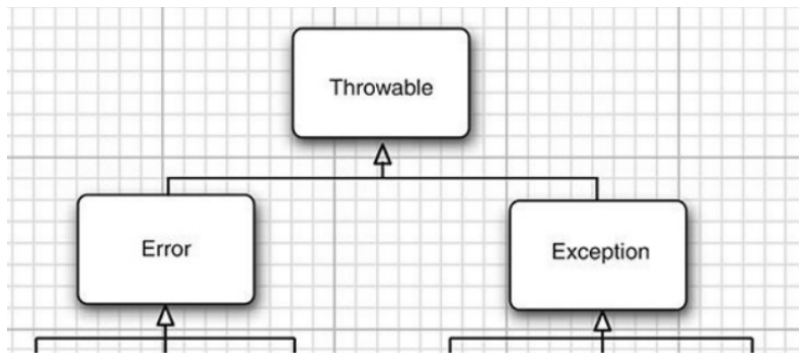
```
Path dir = Paths.get(workingDir);

try(Stream<Path> entries = Files.walk(dir)){
    entries.filter(Files::isRegularFile).forEach(System.out::println);
}
```

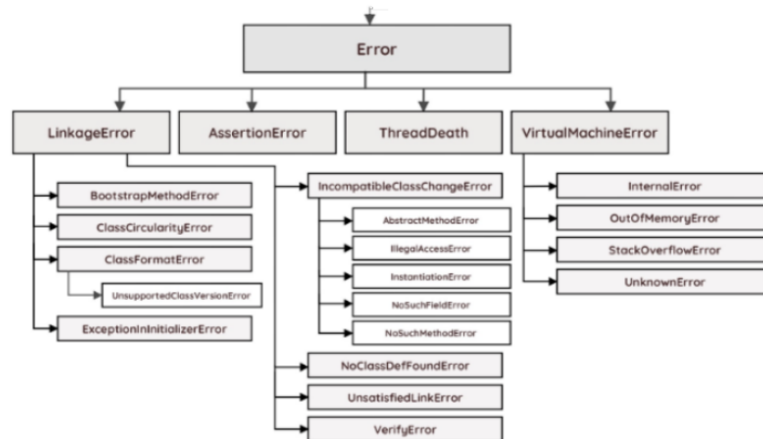


异常处理 Exception Handling

- 异常会打断程序的正常流程。
- Java 异常体系结构：【只有 Throwable 或它的子类才能作为异常处理对象】



- 可以通过 catch 关键字捕获
- **Error（错误）：**



- Error（错误）类体系描述了 Java 运行时系统内部的错误和资源耗尽等异常情况。
- 这些错误表示非常严重的问题，合理的应用程序一般不应该尝试去捕获。
- 比如：内存溢出错误（**OutOfMemoryError**）、栈溢出错误（**StackOverflowError**）等。
- 错误一般是 JVM 在遇到灾难性（fatal）的场景时抛出的，应用程序无法从这类错误中恢复。
- 递归导致栈溢出的异常触发和异常堆栈信息 **StackOverflowError**

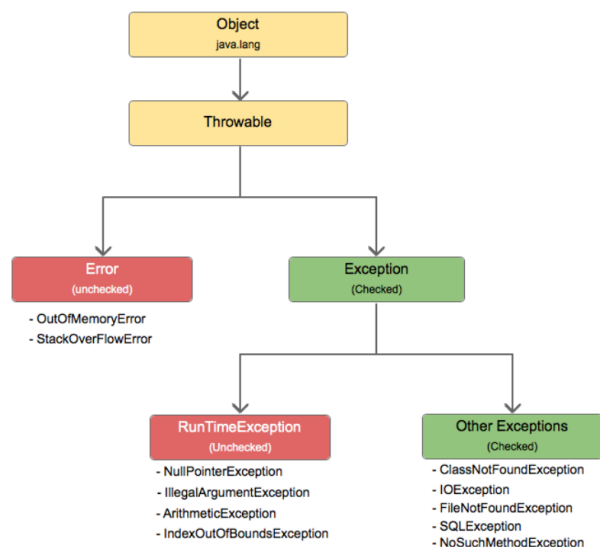
```

public void foo(String s)
{
    foo(s);
}
  
```

```

Exception in thread "main" java.lang.StackOverflowError
    at examples.foo(examples.java:58)
    at examples.foo(examples.java:58)
    at examples.foo(examples.java:58)
    at examples.foo(examples.java:58)
    at examples.foo(examples.java:58)
    at examples.foo(examples.java:58)
    at examples.foo(examples.java:58)
  
```

- **Exception（异常）**



- Exception 表示一种合理的应用程序可能希望捕获并处理的异常情况。
- RuntimeException 及其子类是未检查异常（unchecked exception）。
- 其他异常为已检查异常（checked exception），可理解为由编译器进行检查。

- **Checked Exceptions（已检查异常）**

- 已检查异常不能在编译时被忽略。
- 编译器会强制程序员处理这些异常。
- 处理已检查异常的两种方式：catch（捕获）、throw（向上传递）。

```

public void processFile() {
    File text = new File("C:/test.txt");
    Scanner s = new Scanner(text);
}

```

Unhandled exception type FileNotFoundException
2 quick fixes available:
Add throws declaration
Surround with try/catch
Press 'F2' for focus

```

public void processFile() {
    File text = new File("C:/test.txt");
    try {
        Scanner s = new Scanner(text);
    } catch (FileNotFoundException e) {
        System.out.println("Cannot find file xxx.");
    }
}

public void processFile() throws FileNotFoundException {
    File text = new File("C:/test.txt");
    Scanner s = new Scanner(text);
}

```

- **异常的传播过程（throw过程）**

- 当程序中的某个方法（比如F()）产生异常（throw MyException），这个异常并不是只能在F()里被处理。它会一路“往回传”，沿着方法调用链（F→E→D→C→B→A）往前寻找有没有合适的catch语句来捕获它。

- **寻找匹配的catch处理器（catch过程）**

- JVM（虚拟机）会沿着调用栈从异常发生的位置开始，向上逐层查找，直到找到某一层有合适的catch块来处理这个异常，如果找到了，就在那里被捕获。如果整个调用链都没有catch到，JVM会接管，导致程序终止。

- **The finally Clause（finally语句块）**

- 当你的代码抛出异常时，剩下的代码就不会继续执行了。
- 如果剩余的代码中包含一些关键操作（比如关闭资源），这时就会出现问
题。
- 应将这些关键操作放到 finally 语句块里，finally 块总是会被执行。

```
Scanner scanner = null;
try {
    scanner = new Scanner(new File("test.txt"));
    while (scanner.hasNext()) {
        System.out.println(scanner.nextLine());
    }
} catch (FileNotFoundException e) {
    e.printStackTrace();
} finally {
    if (scanner != null) {
        scanner.close();
    }
}
```

- 不管是否发生异常，finally 里的 scanner.close() 都会被调用，确保资源被正确释放。

- **try-with-resources** 语句可以确保某个资源（如 InputStream、数据库连接等）在使用完后会自动关闭。

```
try (var in = new Scanner(new FileInputStream("..."));
     var out = new PrintWriter("out.txt", StandardCharsets.UTF_8)) {
    while (in.hasNext())
        out.println(in.next().toUpperCase());
}
```

无论 try 块如何结束（正常退出或发生异常），in 和 out 都会被关闭。

【解释】这再次强调了 try-with-resources 的优势：只要用这种写法括起来的资源，无论是否遇到异常，程序块退出时，相关资源都会自动关闭，避免了资源泄漏的问题，是推荐的写法。